Table of Contents

8.3 Customizing Git - Git Hooks . . . 3

Git Hooks . . . 3

Installing a Hook . . . 3

Client-Side Hooks . . . 3

Committing-Workflow Hooks . . . 3

Other Client Hooks . . . 4

Server-Side Hooks . . . 5

Hook examples . . . 6

`applypatch-msg` . . . 6

`commit-msg` . . . 7

`fsmonitor-watchman` . . . 8

`post-update` . . . 12

`post-receive` . . . 13

`pre-applypatch` . . . 14

`pre-commit` . . . 15

`pre-merge-commit` . . . 17

`pre-push` . . . 18

`pre-rebase` . . . 20

`pre-receive` . . . 24

`prepare-commit-msg` . . . 25

`push-to-checkout` . . . 26

`sendemail-validate` . . . 28

`update` . . . 30

List of hook examples

Hook 1 applypatch-msg 6
Hook 2 commit-msg 7
Hook 3 fsmonitor-watchman 8
Hook 4 post-update 12
Hook 5 post-receive 13
Hook 6 pre-applypatch 14
Hook 7 pre-commit 15
Hook 8 pre-merge-commit 17
Hook 9 pre-push 18
Hook 10 pre-rebase 20
Hook 11 pre-receive 24
Hook 12 prepare-commit-msg 25
Hook 13 push-to-checkout 26
Hook 14 sendemail-validate 28
Hook 15 update 30

8.3 Customizing Git - Git Hooks

Git Hooks

Like many other Version Control Systems, Git has a way to fire off custom scripts when certain important actions occur. There are two groups of these hooks: client-side and server-side. Client-side hooks are triggered by operations such as committing and merging, while server-side hooks run on network operations such as receiving pushed commits. You can use these hooks for all sorts of reasons.

Installing a Hook

The hooks are all stored in the hooks subdirectory of the Git directory. In most projects, that's `.git/hooks`. When you initialize a new repository with `git init`, Git populates the hooks directory with a bunch of example scripts, many of which are useful by themselves; but they also document the input values of each script. All the examples are written as shell scripts, with some Perl thrown in, but any properly named executable scripts will work fine – you can write them in Ruby or Python or whatever language you are familiar with. If you want to use the bundled hook scripts, you'll have to rename them; their file names all end with `.sample`.

To enable a hook script, put a file in the hooks subdirectory of your `.git` directory that is named appropriately (without any extension) and is executable. From that point forward, it should be called. We'll cover most of the major hook filenames here.

Client-Side Hooks

There are a lot of client-side hooks. This section splits them into committing-workflow hooks, email-workflow scripts, and everything else.

Note It's important to note that client-side hooks are not copied when you clone a repository. If your intent with these scripts is to enforce a policy, you'll probably want to do that on the server side; see the example in An Example Git-Enforced Policy.

Committing-Workflow Hooks

The first four hooks have to do with the committing process.

The `pre-commit` hook is run first, before you even type in a commit message. It's used to inspect the snapshot that's about to be committed, to see if you've forgotten something, to make sure tests run, or to examine whatever you need to inspect in the code. Exiting non-zero from this hook aborts the commit, although you can bypass it with `git commit --no-verify`. You can do things like check for code style (run lint or something equivalent), check for trailing whitespace (the default hook does exactly this), or check for appropriate documentation on new methods.

The `prepare-commit-msg` hook is run before the commit message editor is fired up but after the default message is created. It lets you edit the default message before the commit author sees it. This hook takes a few parameters: the path to the file that holds the commit message so far, the type of commit, and the commit SHA-1 if this is an amended commit. This hook generally isn't useful for normal commits; rather, it's good for commits where the default message is auto-generated, such as templated commit messages, merge commits, squashed commits, and amended commits. You may use it in conjunction with a commit template to programmatically insert information.

The `commit-msg hook` takes one parameter, which again is the path to a temporary file that contains the commit message written by the developer. If this script exits non-zero, Git aborts the commit process, so you can use it to validate your project state or commit message before allowing a commit to go through. In the last section of this chapter, we'll demonstrate using this hook to check that your commit message is conformant to a required pattern.

After the entire commit process is completed, the `post-commit` hook runs. It doesn't take any parameters, but you can easily get the last commit by running `git log -1 HEAD`. Generally, this script is used for notification or something similar.

Email Workflow Hooks You can set up three client-side hooks for an email-based workflow. They're all invoked by the `git am` command, so if you aren't using that command in your workflow, you can safely skip to the next section. If you're taking patches over email prepared by `git format-patch`, then some of these may be helpful to you.

The first hook that is run is `applypatch-msg` as in Hook 1. It takes a single argument: the name of the temporary file that contains the proposed commit message. Git aborts the patch if this script exits non-zero. You can use this to make sure a commit message is properly formatted, or to normalize the message by having the script edit it in place.

The next hook to run when applying patches via `git am` is `pre-applypatch`. Somewhat confusingly, it is run after the patch is applied but before a commit is made, so you can use it to inspect the snapshot before making the commit. You can run tests or otherwise inspect the working tree with this script. If something is missing or the tests don't pass, exiting non-zero aborts the `git am` script without committing the patch.

The last hook to run during a `git am` operation is `post-applypatch`, which runs after the commit is made. You can use it to notify a group or the author of the patch you pulled in that you've done so. You can't stop the patching process with this script.

Other Client Hooks

The `pre-rebase` hook runs before you rebase anything and can halt the process by exiting non-zero. You can use this hook to disallow rebasing any commits that have already been pushed. The example pre-rebase hook that Git installs does this, although it makes some assumptions that may not match with your workflow.

The `post-rewrite` hook is run by commands that replace commits, such as `git commit --amend` and `git rebase` (though not by git filter-branch). Its single argument is which command triggered the rewrite, and it receives a list of rewrites on stdin. This hook has many of the same uses as the `post-checkout` and `post-merge` hooks.

After you run a successful `git checkout`, the `post-checkout` hook runs; you can use it to set up your working directory properly for your project environment. This may mean moving in large binary files that you don't want source controlled, auto-generating documentation, or something along those lines.

The `post-merge` hook runs after a successful merge command. You can use it to restore data in the working tree that Git can't track, such as permissions data. This hook can likewise validate the presence of files external to Git control that you may want copied in when the working tree changes.

The `pre-push` hook runs during `git push`, after the remote refs have been updated but before any objects have been transferred. It receives the name and location of the remote as parameters, and a list of to-be-updated refs through stdin. You can use it to validate a set of ref updates before a push occurs (a non-zero exit code will abort the push).

Git occasionally does garbage collection as part of its normal operation, by invoking `git gc --auto`. The `pre-auto-gc` hook is invoked just before the garbage collection takes place, and can be used to notify you that this is happening, or to abort the collection if now isn't a good time.

Server-Side Hooks

In addition to the client-side hooks, you can use a couple of important server-side hooks as a system administrator to enforce nearly any kind of policy for your project. These scripts run before and after pushes to the server. The pre hooks can exit non-zero at any time to reject the push as well as print an error message back to the client; you can set up a push policy that's as complex as you wish.

`pre-receive` The first script to run when handling a push from a client is `pre-receive`. It takes a list of references that are being pushed from stdin; if it exits non-zero, none of them are accepted. You can use this hook to do things like make sure none of the updated references are non-fast-forwards, or to do access control for all the refs and files they're modifying with the push.

`update` The update script is very similar to the pre-receive script, except that it's run once for each branch the pusher is trying to update. If the pusher is trying to push to multiple branches, pre-receive runs only once, whereas update runs once per branch they're pushing to. Instead of reading from stdin, this script takes three arguments: the name of the reference (branch), the SHA-1 that reference pointed to before the push, and the SHA-1 the user is trying to push. If the update script exits non-zero, only that reference is rejected; other references can still be updated.

`post-receive` The post-receive hook runs after the entire process is completed and can be used to update other services or notify users. It takes the same stdin data as the pre-receive hook. Examples include emailing a list, notifying a continuous integration server, or updating a ticket-tracking system – you can even parse the commit messages to see if any tickets need to be opened, modified, or closed. This script can't stop the push process, but the client doesn't disconnect until it has completed, so be careful if you try to do anything that may take a long time.

Hook examples

```
% eza -1 .git/hooks
applypatch-msg.sample
commit-msg.sample
fsmonitor-watchman.sample
post-update.sample
post-receive.sample
pre-applypatch.sample
pre-commit.sample
pre-merge-commit.sample
pre-push.sample
pre-rebase.sample
pre-receive.sample
prepare-commit-msg.sample
push-to-checkout.sample
sendemail-validate.sample
update.sample
```

applypatch-msg

```
cat .git/hooks/applypatch-msg.sample
#!/bin/sh
#
# An example hook script to check the commit log message taken by
# applypatch from an e-mail message.
#
# The hook should exit with non-zero status after issuing an
# appropriate message if it wants to stop the commit.  The hook is
# allowed to edit the commit message file.
#
# To enable this hook, rename this file to "applypatch-msg".

. git-sh-setup
commitmsg="$(git rev-parse --git-path hooks/commit-msg)"
test -x "$commitmsg" && exec "$commitmsg" ${1+"$@"}
:
```

Hook 1: applypatch-msg

commit-msg

```
cat .git/hooks/commit-msg.sample
#!/bin/sh
#
# An example hook script to check the commit log message.
# Called by "git commit" with one argument, the name of the file
# that has the commit message.  The hook should exit with non-zero
# status after issuing an appropriate message if it wants to stop the
# commit.  The hook is allowed to edit the commit message file.
#
# To enable this hook, rename this file to "commit-msg".

# Uncomment the below to add a Signed-off-by line to the message.
# Doing this in a hook is a bad idea in general, but the prepare-commit-msg
# hook is more suited to it.
#
# SOB=$(git var GIT_AUTHOR_IDENT | sed -n 's/^\(.*>\).*$/Signed-off-by: \1/p')
# grep -qs "^$SOB" "$1" || echo "$SOB" >> "$1"

# This example catches duplicate Signed-off-by lines.

test "" = "$(grep '^Signed-off-by: ' "$1" |
   sort | uniq -c | sed -e '/^[   ]*1[  ]/d')" || {
  echo >&2 Duplicate Signed-off-by lines.
  exit 1
}
```

Hook 2: commit-msg

fsmonitor-watchman

```
cat .git/hooks/fsmonitor-watchman.sample
#!/usr/bin/perl

use strict;
use warnings;
use IPC::Open2;

# An example hook script to integrate Watchman
# (https://facebook.github.io/watchman/) with git to speed up detecting
# new and modified files.
#
# The hook is passed a version (currently 2) and last update token
# formatted as a string and outputs to stdout a new update token and
# all files that have been modified since the update token. Paths must
# be relative to the root of the working tree and separated by a single NUL.
#
# To enable this hook, rename this file to "query-watchman" and set
# 'git config core.fsmonitor .git/hooks/query-watchman'
#
my ($version, $last_update_token) = @ARGV;

# Uncomment for debugging
# print STDERR "$0 $version $last_update_token\n";

# Check the hook interface version
if ($version ne 2) {
  die "Unsupported query-fsmonitor hook version '$version'.\n" .
      "Falling back to scanning...\n";
}

my $git_work_tree = get_working_dir();

my $retry = 1;

my $json_pkg;
eval {
  require JSON::XS;
  $json_pkg = "JSON::XS";
  1;
} or do {
  require JSON::PP;
  $json_pkg = "JSON::PP";
};

launch_watchman();
```

```
sub launch_watchman {
  my $o = watchman_query();
  if (is_work_tree_watched($o)) {
    output_result($o->{clock}, @{$o->{files}});
  }
}

sub output_result {
  my ($clockid, @files) = @_;

  # Uncomment for debugging watchman output
  # open (my $fh, ">", ".git/watchman-output.out");
  # binmode $fh, ":utf8";
  # print $fh "$clockid\n@files\n";
  # close $fh;

  binmode STDOUT, ":utf8";
  print $clockid;
  print "\0";
  local $, = "\0";
  print @files;
}

sub watchman_clock {
  my $response = qx/watchman clock "$git_work_tree"/;
  die "Failed to get clock id on '$git_work_tree'.\n" .
    "Falling back to scanning...\n" if $? != 0;

  return $json_pkg->new->utf8->decode($response);
}

sub watchman_query {
  my $pid = open2(\*CHLD_OUT, \*CHLD_IN, 'watchman -j --no-pretty')
  or die "open2() failed: $!\n" .
  "Falling back to scanning...\n";

  # In the query expression below we're asking for names of files that
  # changed since $last_update_token but not from the .git folder.
  #
  # To accomplish this, we're using the "since" generator to use the
  # recency index to select candidate nodes and "fields" to limit the
  # output to file names only. Then we're using the "expression" term to
  # further constrain the results.
  my $last_update_line = "";
  if (substr($last_update_token, 0, 1) eq "c") {
```

```
    $last_update_token = "\"$last_update_token\"";
    $last_update_line = qq[\n"since": $last_update_token,];
  }
  my $query = <<" END";
    ["query", "$git_work_tree", {$last_update_line
      "fields": ["name"],
      "expression": ["not", ["dirname", ".git"]]
    }]
  END

  # Uncomment for debugging the watchman query
  # open (my $fh, ">", ".git/watchman-query.json");
  # print $fh $query;
  # close $fh;

  print CHLD_IN $query;
  close CHLD_IN;
  my $response = do {local $/; <CHLD_OUT>};

  # Uncomment for debugging the watch response
  # open ($fh, ">", ".git/watchman-response.json");
  # print $fh $response;
  # close $fh;

  die "Watchman: command returned no output.\n" .
  "Falling back to scanning...\n" if $response eq "";
  die "Watchman: command returned invalid output: $response\n" .
  "Falling back to scanning...\n" unless $response =~ /^\{/;

  return $json_pkg->new->utf8->decode($response);
}

sub is_work_tree_watched {
  my ($output) = @_;
  my $error = $output->{error};
  if ($retry > 0 and $error and $error =~ m/unable to resolve root .* directory
(.*) is not watched/) {
    $retry--;
    my $response = qx/watchman watch "$git_work_tree"/;
    die "Failed to make watchman watch '$git_work_tree'.\n" .
        "Falling back to scanning...\n" if $? != 0;
    $output = $json_pkg->new->utf8->decode($response);
    $error = $output->{error};
    die "Watchman: $error.\n" .
    "Falling back to scanning...\n" if $error;
```

```
    # Uncomment for debugging watchman output
    # open (my $fh, ">", ".git/watchman-output.out");
    # close $fh;

    # Watchman will always return all files on the first query so
    # return the fast "everything is dirty" flag to git and do the
    # Watchman query just to get it over with now so we won't pay
    # the cost in git to look up each individual file.
    my $o = watchman_clock();
    $error = $output->{error};

    die "Watchman: $error.\n" .
    "Falling back to scanning...\n" if $error;

    output_result($o->{clock}, ("/"));
    $last_update_token = $o->{clock};

    eval { launch_watchman() };
    return 0;
  }

  die "Watchman: $error.\n" .
  "Falling back to scanning...\n" if $error;

  return 1;
}

sub get_working_dir {
  my $working_dir;
  if ($^O =~ 'msys' || $^O =~ 'cygwin') {
    $working_dir = Win32::GetCwd();
    $working_dir =~ tr/\\/\//;
  } else {
    require Cwd;
    $working_dir = Cwd::cwd();
  }

  return $working_dir;
}
```

Hook 3: fsmonitor-watchman

post-update

```
cat .git/hooks/post-update.sample
#!/bin/sh
#
# An example hook script to prepare a packed repository for use over
# dumb transports.
#
# To enable this hook, rename this file to "post-update".

exec git update-server-info
```

Hook 4: post-update

post-receive

```
cat .git/hooks/post-receive.sample
#!/bin/sh

#!/bin/sh
#
# An example hook script to extract a worktree to a given directory upon received
#
# Requires git installed on the server side.
#
# To enable this hook, rename this file to "post-receive".

git –work-tree=/var/www/tld.domain.www/ –git-dir=/opt/local/development/project/
checkout -f
```

Hook 5: post-receive

pre-applypatch

```
cat .git/hooks/pre-applypatch.sample
#!/bin/sh
#
# An example hook script to verify what is about to be committed
# by applypatch from an e-mail message.
#
# The hook should exit with non-zero status after issuing an
# appropriate message if it wants to stop the commit.
#
# To enable this hook, rename this file to "pre-applypatch".

. git-sh-setup
precommit="$(git rev-parse --git-path hooks/pre-commit)"
test -x "$precommit" && exec "$precommit" ${1+"$@"}
:
```

Hook 6: pre-applypatch

pre-commit

```
cat .git/hooks/pre-commit.sample
#!/bin/sh
#
# An example hook script to verify what is about to be committed.
# Called by "git commit" with no arguments.  The hook should
# exit with non-zero status after issuing an appropriate message if
# it wants to stop the commit.
#
# To enable this hook, rename this file to "pre-commit".

if git rev-parse --verify HEAD >/dev/null 2>&1
then
  against=HEAD
else
  # Initial commit: diff against an empty tree object
  against=$(git hash-object -t tree /dev/null)
fi

# If you want to allow non-ASCII filenames set this variable to true.
allownonascii=$(git config --type=bool hooks.allownonascii)

# Redirect output to stderr.
exec 1>&2

# Cross platform projects tend to avoid non-ASCII filenames; prevent
# them from being added to the repository. We exploit the fact that the
# printable range starts at the space character and ends with tilde.
if [ "$allownonascii" != "true" ] &&
  # Note that the use of brackets around a tr range is ok here, (it's
  # even required, for portability to Solaris 10's /usr/bin/tr), since
  # the square bracket bytes happen to fall in the designated range.
  test $(git diff-index --cached --name-only --diff-filter=A -z $against |
    LC_ALL=C tr -d '[ -~]\0' | wc -c) != 0
then
  cat <<\EOF
Error: Attempt to add a non-ASCII file name.

This can cause problems if you want to work with people on other platforms.

To be portable it is advisable to rename the file.

If you know what you are doing you can disable this check using:

  git config hooks.allownonascii true
EOF
```

```
  exit 1
fi

# If there are whitespace errors, print the offending file names and fail.
exec git diff-index --check --cached $against --
```

Hook 7: pre-commit

pre-merge-commit

```
cat .git/hooks/pre-merge-commit.sample
#!/bin/sh
#
# An example hook script to verify what is about to be committed.
# Called by "git merge" with no arguments.  The hook should
# exit with non-zero status after issuing an appropriate message to
# stderr if it wants to stop the merge commit.
#
# To enable this hook, rename this file to "pre-merge-commit".

. git-sh-setup
test -x "$GIT_DIR/hooks/pre-commit" &&
        exec "$GIT_DIR/hooks/pre-commit"
:
```

Hook 8: pre-merge-commit

pre-push

```
cat .git/hooks/pre-push.sample
#!/bin/sh

# An example hook script to verify what is about to be pushed.  Called by "git
# push" after it has checked the remote status, but before anything has been
# pushed.  If this script exits with a non-zero status nothing will be pushed.
#
# This hook is called with the following parameters:
#
# $1 -- Name of the remote to which the push is being done
# $2 -- URL to which the push is being done
#
# If pushing without using a named remote those arguments will be equal.
#
# Information about the commits which are being pushed is supplied as lines to
# the standard input in the form:
#
#   <local ref> <local oid> <remote ref> <remote oid>
#
# This sample shows how to prevent push of commits where the log message starts
# with "WIP" (work in progress).

remote="$1"
url="$2"

zero=$(git hash-object --stdin </dev/null | tr '[0-9a-f]' '0')

while read local_ref local_oid remote_ref remote_oid
do
  if test "$local_oid" = "$zero"
  then
    # Handle delete
    :
  else
    if test "$remote_oid" = "$zero"
    then
      # New branch, examine all commits
      range="$local_oid"
    else
      # Update to existing branch, examine new commits
      range="$remote_oid..$local_oid"
    fi

    # Check for WIP commit
    commit=$(git rev-list -n 1 --grep '^WIP' "$range")
```

```
    if test -n "$commit"
    then
      echo >&2 "Found WIP commit in $local_ref, not pushing"
      exit 1
    fi
  fi
done

exit 0
```

Hook 9: pre-push

pre-rebase

```
cat .git/hooks/pre-rebase.sample
#!/bin/sh
#
# Copyright (c) 2006, 2008 Junio C Hamano
#
# The "pre-rebase" hook is run just before "git rebase" starts doing
# its job, and can prevent the command from running by exiting with
# non-zero status.
#
# The hook is called with the following parameters:
#
# $1 -- the upstream the series was forked from.
# $2 -- the branch being rebased (or empty when rebasing the current branch).
#
# This sample shows how to prevent topic branches that are already
# merged to 'next' branch from getting rebased, because allowing it
# would result in rebasing already published history.

publish=next
basebranch="$1"
if test "$#" = 2
then
  topic="refs/heads/$2"
else
  topic=`git symbolic-ref HEAD` ||
  exit 0 ;# we do not interrupt rebasing detached HEAD
fi

case "$topic" in
refs/heads/??/*)
  ;;
*)
  exit 0 ;# we do not interrupt others.
  ;;
esac

# Now we are dealing with a topic branch being rebased
# on top of master.  Is it OK to rebase it?

# Does the topic really exist?
git show-ref -q "$topic" || {
  echo >&2 "No such branch $topic"
  exit 1
}
```

```
# Is topic fully merged to master?
not_in_master=`git rev-list --pretty=oneline ^master "$topic"`
if test -z "$not_in_master"
then
  echo >&2 "$topic is fully merged to master; better remove it."
  exit 1 ;# we could allow it, but there is no point.
fi

# Is topic ever merged to next?  If so you should not be rebasing it.
only_next_1=`git rev-list ^master "^$topic" ${publish} | sort`
only_next_2=`git rev-list ^master           ${publish} | sort`
if test "$only_next_1" = "$only_next_2"
then
  not_in_topic=`git rev-list "^$topic" master`
  if test -z "$not_in_topic"
  then
    echo >&2 "$topic is already up to date with master"
    exit 1 ;# we could allow it, but there is no point.
  else
    exit 0
  fi
else
  not_in_next=`git rev-list --pretty=oneline ^${publish} "$topic"`
  /usr/bin/perl -e '
    my $topic = $ARGV[0];
    my $msg = "* $topic has commits already merged to public branch:\n";
    my (%not_in_next) = map {
      /^([0-9a-f]+) /;
      ($1 => 1);
    } split(/\n/, $ARGV[1]);
    for my $elem (map {
        /^([0-9a-f]+) (.*)$/;
        [$1 => $2];
      } split(/\n/, $ARGV[2])) {
      if (!exists $not_in_next{$elem->[0]}) {
        if ($msg) {
          print STDERR $msg;
          undef $msg;
        }
        print STDERR " $elem->[1]\n";
      }
    }
  ' "$topic" "$not_in_next" "$not_in_master"
  exit 1
fi
```

```
<<\DOC_END

This sample hook safeguards topic branches that have been
published from being rewound.

The workflow assumed here is:

 * Once a topic branch forks from "master", "master" is never
   merged into it again (either directly or indirectly).

 * Once a topic branch is fully cooked and merged into "master",
   it is deleted.  If you need to build on top of it to correct
   earlier mistakes, a new topic branch is created by forking at
   the tip of the "master".  This is not strictly necessary, but
   it makes it easier to keep your history simple.

 * Whenever you need to test or publish your changes to topic
   branches, merge them into "next" branch.

The script, being an example, hardcodes the publish branch name
to be "next", but it is trivial to make it configurable via
$GIT_DIR/config mechanism.

With this workflow, you would want to know:

(1) ... if a topic branch has ever been merged to "next".  Young
    topic branches can have stupid mistakes you would rather
    clean up before publishing, and things that have not been
    merged into other branches can be easily rebased without
    affecting other people.  But once it is published, you would
    not want to rewind it.

(2) ... if a topic branch has been fully merged to "master".
    Then you can delete it.  More importantly, you should not
    build on top of it -- other people may already want to
    change things related to the topic as patches against your
    "master", so if you need further changes, it is better to
    fork the topic (perhaps with the same name) afresh from the
    tip of "master".

Let's look at this example:

       o---o---o---o---o---o---o---o---o---o "next"
      /       /           /           /
     /   a---a---b A     /           /
    /   /               /           /
```

```
         /   /   c---c---c---c B         /
        /   /   /             \         /
       /   /   /   b---b C     \       /
      /   /   /   /             \     /
    ---o---o---o---o---o---o---o---o---o---o---o "master"


A, B and C are topic branches.

 * A has one fix since it was merged up to "next".

 * B has finished.  It has been fully merged up to "master" and "next",
   and is ready to be deleted.

 * C has not merged to "next" at all.

We would want to allow C to be rebased, refuse A, and encourage
B to be deleted.

To compute (1):

  git rev-list ^master ^topic next
  git rev-list ^master        next

  if these match, topic has not merged in next at all.

To compute (2):

  git rev-list master..topic

  if this is empty, it is fully merged to "master".

DOC_END
```

Hook 10: pre-rebase

pre-receive

```
cat .git/hooks/pre-receive.sample
#!/bin/sh
#
# An example hook script to make use of push options.
# The example simply echoes all push options that start with 'echoback='
# and rejects all pushes when the "reject" push option is used.
#
# To enable this hook, rename this file to "pre-receive".

if test -n "$GIT_PUSH_OPTION_COUNT"
then
  i=0
  while test "$i" -lt "$GIT_PUSH_OPTION_COUNT"
  do
    eval "value=\$GIT_PUSH_OPTION_$i"
    case "$value" in
    echoback=*)
      echo "echo from the pre-receive-hook: ${value#*=}" >&2
      ;;
    reject)
      exit 1
    esac
    i=$((i + 1))
  done
fi
```

Hook 11: pre-receive

prepare-commit-msg

```
cat .git/hooks/prepare-commit-msg.sample
#!/bin/sh
#
# An example hook script to prepare the commit log message.
# Called by "git commit" with the name of the file that has the
# commit message, followed by the description of the commit
# message's source.  The hook's purpose is to edit the commit
# message file.  If the hook fails with a non-zero status,
# the commit is aborted.
#
# To enable this hook, rename this file to "prepare-commit-msg".

# This hook includes three examples. The first one removes the
# "# Please enter the commit message..." help message.
#
# The second includes the output of "git diff --name-status -r"
# into the message, just before the "git status" output.  It is
# commented because it doesn't cope with --amend or with squashed
# commits.
#
# The third example adds a Signed-off-by line to the message, that can
# still be edited.  This is rarely a good idea.

COMMIT_MSG_FILE=$1
COMMIT_SOURCE=$2
SHA1=$3

/usr/bin/perl -i.bak -ne 'print unless(m/^. Please enter the commit message/..m/
^#$/)' "$COMMIT_MSG_FILE"

# case "$COMMIT_SOURCE,$SHA1" in
#  ,|template,)
#    /usr/bin/perl -i.bak -pe '
#       print "\n" . `git diff --cached --name-status -r`
#    if /^#/ && $first++ == 0' "$COMMIT_MSG_FILE" ;;
#  *) ;;
# esac

# SOB=$(git var GIT_COMMITTER_IDENT | sed -n 's/^\(.*>\).*$/Signed-off-by: \1/p')
# git interpret-trailers --in-place --trailer "$SOB" "$COMMIT_MSG_FILE"
# if test -z "$COMMIT_SOURCE"
# then
#   /usr/bin/perl -i.bak -pe 'print "\n" if !$first_line++' "$COMMIT_MSG_FILE"
# fi
```

Hook 12: prepare-commit-msg

push-to-checkout

```
cat .git/hooks/push-to-checkout.sample
#!/bin/sh

# An example hook script to update a checked-out tree on a git push.
#
# This hook is invoked by git-receive-pack(1) when it reacts to git
# push and updates reference(s) in its repository, and when the push
# tries to update the branch that is currently checked out and the
# receive.denyCurrentBranch configuration variable is set to
# updateInstead.
#
# By default, such a push is refused if the working tree and the index
# of the remote repository has any difference from the currently
# checked out commit; when both the working tree and the index match
# the current commit, they are updated to match the newly pushed tip
# of the branch. This hook is to be used to override the default
# behaviour; however the code below reimplements the default behaviour
# as a starting point for convenient modification.
#
# The hook receives the commit with which the tip of the current
# branch is going to be updated:
commit=$1

# It can exit with a non-zero status to refuse the push (when it does
# so, it must not modify the index or the working tree).
die () {
  echo >&2 "$*"
  exit 1
}

# Or it can make any necessary changes to the working tree and to the
# index to bring them to the desired state when the tip of the current
# branch is updated to the new commit, and exit with a zero status.
#
# For example, the hook can simply run git read-tree -u -m HEAD "$1"
# in order to emulate git fetch that is run in the reverse direction
# with git push, as the two-tree form of git read-tree -u -m is
# essentially the same as git switch or git checkout that switches
# branches while keeping the local changes in the working tree that do
# not interfere with the difference between the branches.

# The below is a more-or-less exact translation to shell of the C code
# for the default behaviour for git's push-to-checkout hook defined in
# the push_to_deploy() function in builtin/receive-pack.c.
#
```

```
# Note that the hook will be executed from the repository directory,
# not from the working tree, so if you want to perform operations on
# the working tree, you will have to adapt your code accordingly, e.g.
# by adding "cd .." or using relative paths.

if ! git update-index -q --ignore-submodules --refresh
then
  die "Up-to-date check failed"
fi

if ! git diff-files --quiet --ignore-submodules --
then
  die "Working directory has unstaged changes"
fi

# This is a rough translation of:
#
#   head_has_history() ? "HEAD" : EMPTY_TREE_SHA1_HEX
if git cat-file -e HEAD 2>/dev/null
then
  head=HEAD
else
  head=$(git hash-object -t tree --stdin </dev/null)
fi

if ! git diff-index --quiet --cached --ignore-submodules $head --
then
  die "Working directory has staged changes"
fi

if ! git read-tree -u -m "$commit"
then
  die "Could not update working tree to new HEAD"
fi
```

Hook 13: push-to-checkout

sendemail-validate

```
cat .git/hooks/sendemail-validate.sample
#!/bin/sh

# An example hook script to validate a patch (and/or patch series) before
# sending it via email.
#
# The hook should exit with non-zero status after issuing an appropriate
# message if it wants to prevent the email(s) from being sent.
#
# To enable this hook, rename this file to "sendemail-validate".
#
# By default, it will only check that the patch(es) can be applied on top of
# the default upstream branch without conflicts in a secondary worktree. After
# validation (successful or not) of the last patch of a series, the worktree
# will be deleted.
#
# The following config variables can be set to change the default remote and
# remote ref that are used to apply the patches against:
#
#   sendemail.validateRemote (default: origin)
#   sendemail.validateRemoteRef (default: HEAD)
#
# Replace the TODO placeholders with appropriate checks according to your
# needs.

validate_cover_letter () {
  file="$1"
  # TODO: Replace with appropriate checks (e.g. spell checking).
  true
}

validate_patch () {
  file="$1"
  # Ensure that the patch applies without conflicts.
  git am -3 "$file" || return
  # TODO: Replace with appropriate checks for this patch
  # (e.g. checkpatch.pl).
  true
}

validate_series () {
  # TODO: Replace with appropriate checks for the whole series
  # (e.g. quick build, coding style checks, etc.).
  true
}
```

```
# main ------------------------------------------------------------------------

if test "$GIT_SENDEMAIL_FILE_COUNTER" = 1
then
  remote=$(git config --default origin --get sendemail.validateRemote) &&
  ref=$(git config --default HEAD --get sendemail.validateRemoteRef) &&
  worktree=$(mktemp --tmpdir -d sendemail-validate.XXXXXXX) &&
  git worktree add -fd --checkout "$worktree" "refs/remotes/$remote/$ref" &&
  git config --replace-all sendemail.validateWorktree "$worktree"
else
  worktree=$(git config --get sendemail.validateWorktree)
fi || {
  echo "sendemail-validate: error: failed to prepare worktree" >&2
  exit 1
}

unset GIT_DIR GIT_WORK_TREE
cd "$worktree" &&

if grep -q "^diff --git " "$1"
then
  validate_patch "$1"
else
  validate_cover_letter "$1"
fi &&

if test "$GIT_SENDEMAIL_FILE_COUNTER" = "$GIT_SENDEMAIL_FILE_TOTAL"
then
  git config --unset-all sendemail.validateWorktree &&
  trap 'git worktree remove -ff "$worktree"' EXIT &&
  validate_series
fi
```

Hook 14: sendemail-validate

update

```
cat .git/hooks/update.sample
#!/bin/sh
#
# An example hook script to block unannotated tags from entering.
# Called by "git receive-pack" with arguments: refname sha1-old sha1-new
#
# To enable this hook, rename this file to "update".
#
# Config
# ------
# hooks.allowunannotated
#   This boolean sets whether unannotated tags will be allowed into the
#   repository.  By default they won't be.
# hooks.allowdeletetag
#   This boolean sets whether deleting tags will be allowed in the
#   repository.  By default they won't be.
# hooks.allowmodifytag
#   This boolean sets whether a tag may be modified after creation. By default
#   it won't be.
# hooks.allowdeletebranch
#   This boolean sets whether deleting branches will be allowed in the
#   repository.  By default they won't be.
# hooks.denycreatebranch
#   This boolean sets whether remotely creating branches will be denied
#   in the repository.  By default this is allowed.
#

# --- Command line
refname="$1"
oldrev="$2"
newrev="$3"

# --- Safety check
if [ -z "$GIT_DIR" ]; then
  echo "Don't run this script from the command line." >&2
  echo " (if you want, you could supply GIT_DIR then run" >&2
  echo "  $0 <ref> <oldrev> <newrev>)" >&2
  exit 1
fi

if [ -z "$refname" -o -z "$oldrev" -o -z "$newrev" ]; then
  echo "usage: $0 <ref> <oldrev> <newrev>" >&2
  exit 1
fi
```

```
# --- Config
allowunannotated=$(git config --type=bool hooks.allowunannotated)
allowdeletebranch=$(git config --type=bool hooks.allowdeletebranch)
denycreatebranch=$(git config --type=bool hooks.denycreatebranch)
allowdeletetag=$(git config --type=bool hooks.allowdeletetag)
allowmodifytag=$(git config --type=bool hooks.allowmodifytag)

# check for no description
projectdesc=$(sed -e '1q' "$GIT_DIR/description")
case "$projectdesc" in
"Unnamed repository"* | "")
  echo "*** Project description file hasn't been set" >&2
  exit 1
  ;;
esac

# --- Check types
# if $newrev is 0000...0000, it's a commit to delete a ref.
zero=$(git hash-object --stdin </dev/null | tr '[0-9a-f]' '0')
if [ "$newrev" = "$zero" ]; then
  newrev_type=delete
else
  newrev_type=$(git cat-file -t $newrev)
fi

case "$refname","$newrev_type" in
  refs/tags/*,commit)
    # un-annotated tag
    short_refname=${refname##refs/tags/}
    if [ "$allowunannotated" != "true" ]; then
      echo "*** The un-annotated tag, $short_refname, is not allowed in this
repository" >&2
      echo "*** Use 'git tag [ -a | -s ]' for tags you want to propagate." >&2
      exit 1
    fi
    ;;
  refs/tags/*,delete)
    # delete tag
    if [ "$allowdeletetag" != "true" ]; then
      echo "*** Deleting a tag is not allowed in this repository" >&2
      exit 1
    fi
    ;;
  refs/tags/*,tag)
    # annotated tag
    if [ "$allowmodifytag" != "true" ] && git rev-parse $refname > /dev/null 2>&1
```

```
    then
      echo "*** Tag '$refname' already exists." >&2
      echo "*** Modifying a tag is not allowed in this repository." >&2
      exit 1
    fi
    ;;
  refs/heads/*,commit)
    # branch
    if [ "$oldrev" = "$zero" -a "$denycreatebranch" = "true" ]; then
      echo "*** Creating a branch is not allowed in this repository" >&2
      exit 1
    fi
    ;;
  refs/heads/*,delete)
    # delete branch
    if [ "$allowdeletebranch" != "true" ]; then
      echo "*** Deleting a branch is not allowed in this repository" >&2
      exit 1
    fi
    ;;
  refs/remotes/*,commit)
    # tracking branch
    ;;
  refs/remotes/*,delete)
    # delete tracking branch
    if [ "$allowdeletebranch" != "true" ]; then
      echo "*** Deleting a tracking branch is not allowed in this repository" >&2
      exit 1
    fi
    ;;
  *)
    # Anything else (is there anything else?)
    echo "*** Update hook: unknown type of update to ref $refname of type
$newrev_type" >&2
    exit 1
    ;;
esac

# --- Finished
exit 0
```

Hook 15: update